

LC 312: Burst Balloons

Authored by: Arpit Raj, LNMIIT Jaipur

Date: September 4, 2026

© All rights reserved.

Samsung, Cisco, Flipkart, PhonePe

Problem Understanding

- Given n balloons.
- Burst i^{th} balloon $\rightarrow$ $coins += arr[i-1] \times arr[i] \times arr[i+1]$.
- If out of bound, don't consider that loon (treat it as 1).

Example Walkthrough

Given Array: **[3, 1, 5, 8]** — need max coins.

```
Burst 1: [3, 1, 5, 8]  $\rightarrow (3 \times 1 \times 5) = 15$   
now [3, 5, 8]  $\rightarrow (3 \times 5 \times 8) = 120$   
now [3, 8]  $\rightarrow (1 \times 3 \times 8) = 24$   
now [8]  $\rightarrow (1 \times 8 \times 1) = 8$ 
```

Total = $15 + 120 + 24 + 8 = 167$

**Instead of asking which balloon to burst first,
ask which balloon is burst last.**

Key Intuition

$$[l-1] \ (\ [l \ . \ . \ . \ k \ . \ . \ . \ r] \) \ [r+1]$$

↑
last

k is the last balloon being burst in ***[l...r]***.

Everything in ***[l...r]*** is burst and neighbour of ***k*** is ***(l-1)*** and ***(r+1)***.

And before bursting last balloon, we need to process:

$$[l \ . \ . \ . \ k-1] \ \text{and} \ [k+1 \ . \ . \ . \ r]$$

Dynamic Programming State

dp[l][r] = max(dp[l][k-1] + dp[k+1][r] + nums[l-1] × nums[k] × nums[r+1])

- **Add 1 on both side**, to handle out of bound multiplication:

```
nums.insert(nums.begin(), 1);
```

```
nums.push_back(1);
```

- ***dp[l][r]*** making start with interval lengths of 1.
- Maximize ***dp[l][r]*** by thinking which balloon to burst last in range of ***l*** to ***r***.

$$\begin{array}{ccc} [l-1] & (\ [l \ . \ . \ . \ k \ . \ . \ . \ r] \) & [r+1] \\ & \swarrow \qquad \qquad \searrow & \\ [l \ . \ . \ . \ k-1] & \text{and} & [k+1 \ . \ . \ . \ r] \end{array}$$

C++ Implementation

```
class Solution {
public:
    int maxCoins(vector<int>& nums) {

        int n = nums.size();
        nums.insert(nums.begin(), 1);
        nums.push_back(1);

        vector<vector<int>> dp(n+2, vector<int>(n+2, 0));

        // dp[l][r] will have max coins by bursting loon in range l to r

        for(int len = 1; len <= n; len++) {
            for(int l = 1; l+len-1 <= n; l++) {
                int r = l+len-1;

                for(int k = l; k <= r; k++) {
                    // last loon to get bursted

                    int curr = nums[k] * nums[l-1] * nums[r+1]; // l to r saare burst ho c
                    curr += dp[l][k-1];
                    curr += dp[k+1][r];

                    dp[l][r] = max(dp[l][r], curr);
                }
            }
        }

        return dp[1][n];
    }
};
```